AU Webshop iFlow Design – Sync & Async

Nov 8, 2026 · @Igor

Purpose and scope

This proposes two template iFlows for the AU webshop integration on SAP Integration Suite: one for the **synchronous** pattern S (shop → BTP → S/4HANA Public Cloud AU, designed on S1 customer prices) and one for the **asynchronous** pattern A (shop → BTP → S/4, designed on A4 order create). Both are built so that S2–S5 and A1–A3 are copies with a different mapping and backend call, not new designs.

Basis: *Australien-Konzeption: Integration Architecture* v0.10.0 (2026-10-27, arc42, target architecture path B / ADR-022) and a read-only review of the non-production tenant `fsm-dev-cpi-hydac` (eu10-003) on 2026-11-08. Pattern B (SAP → shop) reuses the async consumer skeleton and is only referenced here.

What the design guarantees, mapped to the quality goals of the concept:

Quality goal (concept §1.3)	How this design meets it
1 · Sync P95: price < 2 s, 100 prices < 4 s, order simulate < 10 s	No persistence on the sync hot path, parallel S/4 calls, explicit per-call timeouts, fail-fast error contract
2 · No lost orders	Event Mesh → JMS staging with transactional consumption, exponential-backoff retry, explicit dead-letter queue, alert on every dead-lettered message
3 · SAP is the single source of truth	Idempotency check against S/4 before every create; result written back to Magento as order comment
4 · Reusability (EU on-premise, US private cloud)	Contract layer separated from backend layer by ProcessDirect; backend chosen by system key `AU`
5 · Operability	Custom MPL headers, stable error codes, DLQ visible in *Manage Message Queues*, reprocessing flow for operators

Tenant findings that shape the design

The tenant already has a working JMS retry/dead-letter pattern from the US team, and a hard JMS queue budget that AU must respect. Both are taken over rather than reinvented.

Finding (tenant `fsm-dev-cpi-hydac`, read-only, 2026-11-08)	Consequence for AU
44 packages; naming `<REGION>-<AREA>-<Name>` (e.g. `US-SLS-SalesInquiry_to_SAP`, `AU-EDI-Orders850`); iFlow IDs `com.hydac.<REGION>.<AREA>.<name>` (e.g. `com.hydac.US.SLS.get_wsinquiry`)	AU packages as in the concept (`AU-SLS-WebshopSyncAPIs`, `AU-SLS-WebshopToSAP`, `AU-SLS-SAPToWebshop`, `AU-SLS-GenericIS_Artifacts`); iFlow IDs `com.hydac.AU.SLS.*`
`US-SLS-GenericIS_Artifacts` holds `process_JMSQueues`: JMS sender on one shared queue `US_HYDAC_JMS_REPROCESSQ` (retry 10 min, exponential backoff, max 600 min, DLQ flag on, 1 concurrent process) → Content Modifier sets header `ReceiverPath = ${property.interfacename}` → ProcessDirect `/${header.ReceiverPath}`	AU reuses the same dispatcher idea: one dead-letter queue per stage, the owning iFlow named in a property, replay through ProcessDirect
JMS resources: 30 queues maximum, 12 in use (AS2, TPM, EDI, US flows), 9 000 MB capacity, 0.46 MB used, 2 runtime nodes	18 queues are left for AU, US, DE and TPM together. AU gets a fixed budget (proposal: 6 in non-prod, see queue plan) and never one queue per interface
`US-SLS-SalesInquiry_to_SAP` (`get_wsinquiry`): HTTPS → properties → OData request-reply → mapping → XML-to-JSON → exception subprocess with error script	Same skeleton for the sync template, extended by validation, parallel backend calls, timeouts and the RFC 9457 error contract
`AU-EDI-Orders850` shows no artifacts for my user	Probably hidden by the access policy `AP_AU_Developer` (concept §8.11, Q-12). The AU shop packages need the `AU-` prefix so they fall under the same policy, and the policy must grant read to HYDAC Digital and TechDivision

Finding (tenant `fsm-dev-cpi-hydac`, read-only, 2026-11-08)	Consequence for AU
Only `Advanced Event Mesh Adapter` is installed as adapter package; standard Event Mesh is used through the built-in AMQP adapter	Fits D-22 (standard Event Mesh); no extra adapter needed

The concept names the tenants `hydac-is-nonprod` / `hydac-is-prod`; the tenant reviewed is reachable as `fsm-dev-cpi-hydac`. Please confirm this is the non-production tenant meant by the concept (stages DEV and TST).

Design principles for both iFlows

Sync flows fail fast and never persist; async flows persist first and retry with backoff. Everything else is shared.

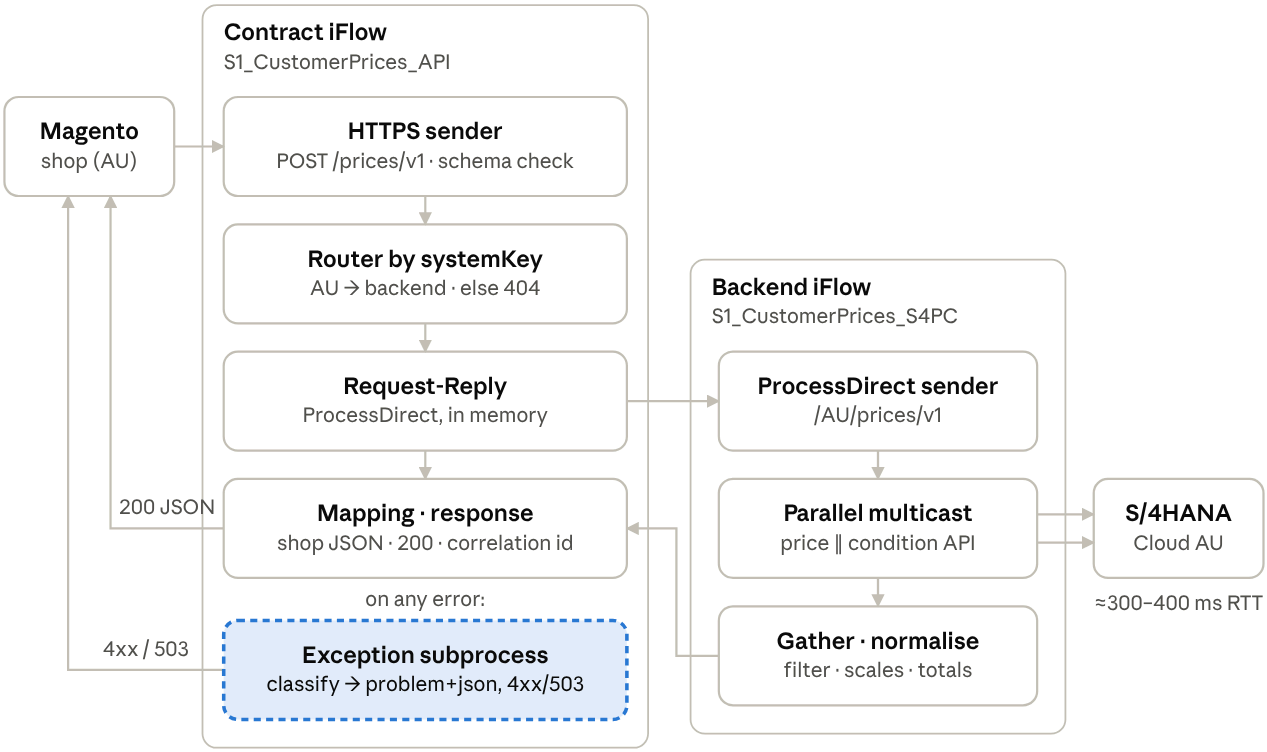
1. **Two layers per interface.** A *contract* iFlow owns the shop-facing REST contract (validation, normalisation, RFC 9457 errors, correlation) and calls a *backend* iFlow per SAP system over ProcessDirect (`/hydac/au/sls/<system>/<interface>/v1`). The backend iFlow composes the released S/4 Public Cloud APIs. EU (on-premise `ZEC_*`) and US later add a backend iFlow, not a new contract (G-3, C-1). For the MVP both layers may live in one iFlow with a local integration process at the seam; the split is then a refactoring, not a redesign.
2. **Routing is configuration.** System key (`AU`), sales area, Magento website/store IDs, endpoints and credential aliases are externalised parameters, set per stage copy (`…_DEV`, `…_TST`) and never hard-coded (D-37, Q-16).
3. **Error classes drive behaviour, not adapters.** One Groovy script (in the shared script collection) classifies every exception into `TRANSIENT` (S/4 or Magento 5xx, 429, connection/timeout, Event Mesh/JMS unavailable), `PERMANENT_TECH` (mapping, 4xx, schema) or `PERMANENT_BUSINESS` (S/4 return type `E`, precondition not met). Only `TRANSIENT` is ever retried.
4. **Lean payloads** (C-5, G-5): only fields the shop uses; large results paged from S/4 and chunked into Magento.
5. **Correlation end-to-end**: `X-Correlation-ID` from the shop (or the CloudEvent `id`) is stored in `SAP_ApplicationID` and returned in every response and comment, so HYDAC Digital and TechDivision can find a message without BTP write access.

6. **Observable by non-developers**: custom MPL header properties (`shopSystem` , `websiteCode` , `entity` , `entityId` , `incrementId` , `errorClass` , `errorCode`) on every run, a custom status on failure, and an Alert Notification Service call on every dead-lettered message.
7. **No secrets in flows**: credentials only as security material (`AU_S4_TST` , `AU_Magento_SHL` , …), shop → BTP via OAuth2 client credentials with a dedicated user role, BTP → S/4 via the communication arrangement's technical user (basic or X.509, DyFlex to confirm).
8. **Idempotent by design**: every write to S/4 or Magento is safe to repeat (lookup-before-create, PUT semantics, Idempotent Process Call keyed by the CloudEvent `id`).

Sync iFlow – S1 customer prices as template

The sync iFlow answers one shop request with one BTP call, keeps nothing persistent on the hot path, and turns every failure into a stable `application/problem+json` answer within the shop's client timeout. No in-flow retry loop: the shop already degrades (list price with notice) and retries on the next page load (concept §8.8); a retry inside the flow would double S/4 load exactly when S/4 is struggling.

S1 sync: one hop, parallel S/4 calls, a JSON error on every failure



S1 sync iFlow · contract layer, backend layer, S/4 calls, error path

The contract iFlow owns the shop-facing endpoint and the error contract; the backend iFlow owns the S/4 calls and runs them in parallel. Every failure, from any step, ends in the exception subprocess and leaves as a `problem+json` body with a 4xx or 503 status.

iFlows (package `AU-SLS-WebshopSyncAPIs`)

Artefact	ID	Role
`CustomerPrices_API_v1`	`com.hydac.AU.SLS.S1_CustomerPrices_API`	Contract layer: HTTPS endpoint `/hydac/au/sls/prices/v1`, validation, routing by system key, response normalisation, error contract
`CustomerPrices_S4PC`	`com.hydac.AU.SLS.S1_CustomerPrices_S4PC`	Backend layer for S/4HANA Public Cloud: parallel calls to the price and condition APIs, scale conversion, condition filtering per country

Artefact	ID	Role
`AU_SLS_Common` (script collection, in `AU-SLS-GenericIS_Artifacts`)	–	`classifyError`, `problemJson`, `mplHeaders`, `jsonSchemaValidate`
`AU_ConditionTypes` (value mapping)	–	Relevant condition types per country/system → shop condition kind (list, net, discount, surcharge, freight, tax)

Steps of the contract iFlow

#	Step	Detail
1	HTTPS sender	`POST /hydac/au/sls/prices/v1`, OAuth2 client credentials, user role `AU_Webshop_Send` (custom role, not `ESBMessaging.send`, so AU shop clients cannot call US endpoints), CSRF off, max body size externalised
2	Content Modifier `Capture context`	Properties `correlationId` (`X-Correlation-ID`, fallback MPL id), `systemKey`, `salesOrg` / `distChannel` / `division`, `customerId`, `materialCount`; `SAP_ApplicationID = correlationId`; MPL custom headers via script
3	Groovy `Validate request`	JSON schema from the OpenAPI contract (D-04); max 100 materials per call; on failure throw with class `PERMANENT_TECH`, code `VALIDATION_FAILED` → HTTP 400
4	Router `by systemKey`	`AU` → ProcessDirect `/hydac/au/sls/AU/prices/v1`; unknown key → 404 `UNKNOWN_SYSTEM`. Future `EU`, `US` branches are new backend iFlows only
5	Request-Reply ProcessDirect	Backend call; the backend returns the normalised internal XML

#	Step	Detail
6	Message mapping `to shop contract`	Internal XML → shop JSON: `listPrice`, `netPrice`, `conditions[]`, `scales[]`, amounts with currency, quantities with unit (concept §8.2)
7	Content Modifier `Response`	`Content-Type: application/json`, `X-Correlation-ID`, `Cache-Control: no-store` (the shop caches, the BTP does not, D-10), HTTP 200
E	Exception subprocess	`classifyError` → `problemJson` body → `CamelHttpResponseCode` per table below → MPL custom status `AU_SYNC_FAILED_<class>` → End (never Error End: the shop must get a JSON body, not a CPI HTML error)

Steps of the backend iFlow (S/4 Public Cloud)

#	Step	Detail
1	ProcessDirect sender	`/hydac/au/sls/AU/prices/v1`
2	Parallel Multicast	Branch A: price API; branch B: condition-record API (exact released APIs per interface document S1 and the DyFlex IDD, Q-01). Parallel so that `n_S4` round trips of 300–400 ms count once (concept §7.3)
3	OData V2/V4 receivers	Credential `AU_S4_<STG>`, `$select` only the fields used, `$filter` on customer + materials + sales area, `$top` paging for > 100 materials; timeout externalised: `s4.timeout.ms = 1200` for S1/S3/S4, `8000` for S2; connection reuse on
4	Gather + Groovy `Normalise`	Join by material; keep only condition types from `AU_ConditionTypes`; sum totals; convert scales to the sales unit
5	End	Internal XML back to the contract flow

Error contract (one `code` per business error, stable across systems; body always RFC 9457 with `type`, `title`, `status`, `detail`, `instance = correlationId`, `code`)

Situation	Class	HTTP	`code`	Shop behaviour (concept §8.8)
Schema or business validation fails	PERMANENT_TECH	400	`VALIDATION_FAILED`	Fix request; no retry
Unknown system key / sales area	PERMANENT_TECH	404	`UNKNOWN_SYSTEM`	Configuration error, alert purplecat
Customer or material not found in S/4	PERMANENT_BUSINESS	404	`NOT_FOUND`	Show list price / message
S/4 returns a business error (`E`)	PERMANENT_BUSINESS	422	`S4_BUSINESS_ERROR` + S/4 message id	Message to user, no retry
S/4 timeout, 5xx, 429, connection refused (maintenance window, C-2)	TRANSIENT	503 + `Retry-After: 30`	`S4_UNAVAILABLE`	S1: list price with notice; S2: hard gate; retry on next request
BTP internal error (mapping, script)	PERMANENT_TECH	500	`INTEGRATION_ERROR`	Alert purplecat; shop degrades as for 503

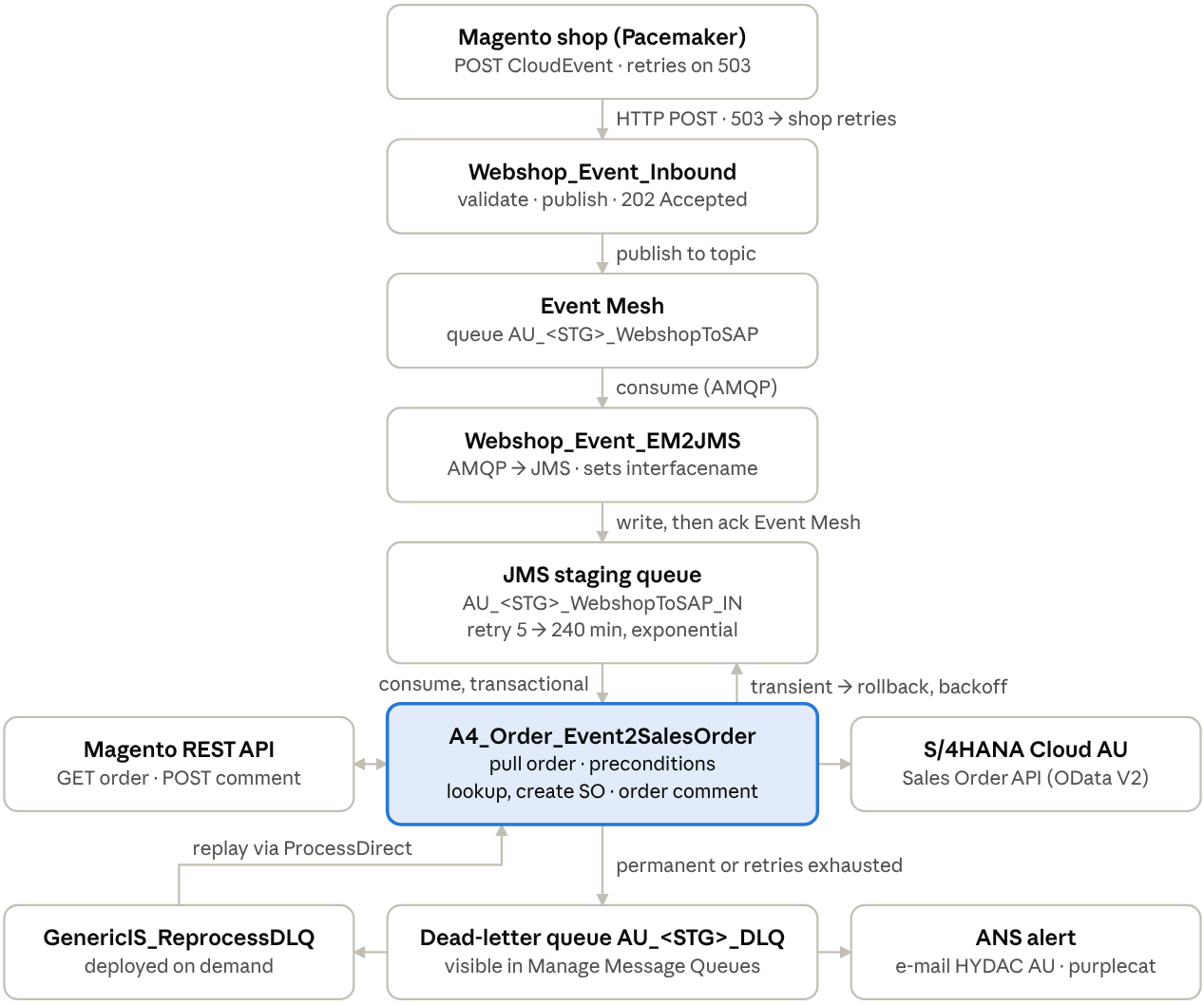
Timeouts and budget for S1 (P95 < 2 s measured at the shop): shop client timeout 2 000 ms (TechDivision), BTP per-S/4-call timeout 1 200 ms, parallel calls, no retry. For S2 (< 10 s): client timeout 10 000 ms, S/4 timeout 8 000 ms. The externalised timeouts are adjusted after the latency and load test (concept §10.2); the structure stays.

What S2–S5 change: S2 replaces the backend by one call to the sales order simulation API (no multicast) and maps the full simulation result; S3 calls the ATP/replacement-time API; S4 pages the order history (`$top/$skip` loop with a Looping Process Call); S5 is open. The contract flow, error contract, validation, routing and monitoring are copied unchanged.

Async chain – A4 order create as template

An order event is persisted three times before anything is attempted against S/4 – by the shop's Pacemaker pipeline, by Event Mesh and by a CI JMS queue – and every attempt is idempotent, so an order is either in S/4 or in the dead-letter queue with an alert; it is never lost (quality goal 2). The chain is three iFlows: a thin generic inbound, a thin Event Mesh → JMS bridge, and the consumer that does the work.

A4 async: persisted three times before S/4; every order ends in S/4 or the DLQ



A4 async chain · inbound, Event Mesh, JMS staging, consumer, dead letter, reprocessing

The order is persisted by the shop pipeline, by Event Mesh and by the JMS staging queue before the consumer touches S/4. Transient failures roll the JMS message back and it is redelivered with backoff; permanent failures and exhausted retries go to the dead-letter queue, raise an alert and can be replayed by an operator.

Why JMS behind Event Mesh. Event Mesh gives the multiplex (one copy per subscriber) that D-38/D-40 need, but its redelivery is immediate and bounded (max redelivery count, then its dead-message queue). During a weekend S/4 maintenance window (C-2) that would exhaust the redeliveries within seconds. The CI JMS sender adapter retries with exponential backoff (as the US team already configures it: 10 min → max 600 min) and runs the consumer in a transaction, so the message is only acknowledged once S/4 has accepted it. The cost is one extra, tiny bridge message per event. Note that the JMS adapter's own *Dead-Letter Queue* flag is not a business dead-letter queue: it only parks a message that crashed the worker node twice (visible as `JmsDeadLetter` in the Message Locks monitor, see SAP blog). Ordinary failures would otherwise retry until the message expires, which is why this design dead-letters explicitly after a retry budget. Event Mesh queues carry `maxRedeliveryCount` (default 0) and `deadMsgQueue` (default none), which must be set when the queues are created (SAP community).

iFlows (package `AU-SLS-WebshopToSAP`, shared ones in `AU-SLS-GenericIS_Artifacts`)

Artefact	ID	Role
`Webshop_Event_Inbound`	`com.hydac.AU.SLS.Webshop_Event_Inbound`	Generic inbound (D-38): `POST /hydac/au/sls/events/v1`, validates the CloudEvent, publishes to Event Mesh topic `ci/au/shop/<entity>/v1/<event>`, answers `202`
`Webshop_Event_EM2JMS`	`com.hydac.AU.SLS.Webshop_Event_EM2JMS`	Bridge: AMQP sender on EM queue `AU_<STG>_WebshopToSAP` → JMS receiver `AU_<STG>_WebshopToSAP_IN` with `interfacename` property set from the event type
`Order_Event2SalesOrder`	`com.hydac.AU.SLS.A4_Order_Event2SalesOrder`	Consumer: JMS sender `AU_<STG>_WebshopToSAP_IN` (filtered by `interfacename`, see note) → pull order from Magento → preconditions → idempotency check → create sales order → order comment
`GenericIS_ReprocessDLQ`	`com.hydac.AU.SLS.GenericIS_ReprocessDLQ`	Copy of the US `process_JMSQueues` pattern: JMS sender on `AU_<STG>_DLQ` → ProcessDirect `/${property.interfacename}/reprocess`; **undeployed by default**, deployed by an operator to replay after the root cause is fixed

Note on the shared IN queue: the JMS sender adapter cannot filter by property, so one queue per consumer is the clean design (A1–A4 = 4 queues per stage) and the shared queue plus a dispatcher is the budget-saving one. With 18 queues left in the tenant I propose the **dispatcher variant**: the bridge writes to one IN queue per stage; the consumer side is one small dispatcher iFlow (`Webshop_Event_Dispatch`, JMS sender → ProcessDirect `/hydac/au/sls/<interfacename>/v1`) and A1–A4 start with a

ProcessDirect sender. The JMS transaction spans the ProcessDirect call, so retry and dead-letter behave as if each consumer had its own queue. If HYDAC IT grants AU a larger queue budget, switch to one IN queue per consumer; nothing else changes.

Steps of the generic inbound iFlow

#	Step	Detail
1	HTTPS sender	`POST /hydac/au/sls/events/v1`, OAuth2 client credentials, role `AU_Webshop_Send`
2	Groovy `Validate CloudEvent`	`specversion`, `type` in the allowed list (`company
3	Content Modifier	Properties `eventType`, `shopSystem`, `websiteCode`, `entity`, `entityId`, `correlationId = id`; `SAP_ApplicationID = id`; MPL headers
4	AMQP receiver (Event Mesh)	Topic `ci/au/shop/<entity>/v1/<event>`, message client `AU_<STG>_CI`; body = the CloudEvent as received; headers carried as AMQP application properties
5	Response `202 Accepted`	Body `{ "id": …, "correlationId": … }`
E	Exception subprocess	Event Mesh unreachable → **503** with `Retry-After` so that Pacemaker retries (concept §8.4 leg 1); validation → 400 with problem+json. No JMS fallback here: the shop is the retry owner for leg 1

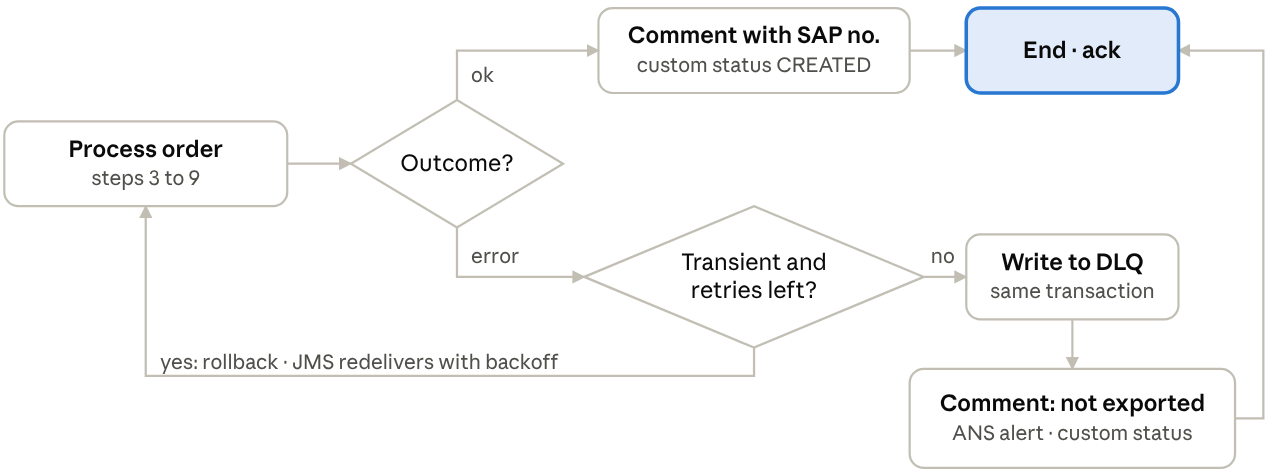
Steps of the consumer iFlow (A4) – integration process with *Transaction handling: Required for JMS*

#	Step	Detail
1	JMS sender (via dispatcher)	Queue `AU_<STG>_WebshopToSAP_IN`, retry interval 5 min, exponential backoff on, max retry interval 240 min, concurrent processes 1 per node (orders are low volume; raise for B flows), DLQ flag on (protects against node crashes)

#	Step	Detail
2	Content Modifier `Restore context`	Properties from the persisted event (`shopSystem` , `incrementId` , …), `retryCount = ${header.SAPJMSRetries}` (0 on first attempt), `maxRetries` externalised (default 8 ≈ 20 h with the backoff above)
3	Request-Reply HTTP `GET Magento order`	`GET /rest/<store>/V1/orders/<entityId>` with credential `AU_Magento_<SHOP>` chosen from `shopSystem` (routing by configuration); 404 → PERMANENT_TECH
4	Groovy `Preconditions` (D-14, D-32)	Customer has SAP number and is approved, ship-to has SAP partner number, order is paid if credit card (`prepaid` , PSP, transaction id present); on failure PERMANENT_BUSINESS `ORDER_PRECONDITION_FAILED`
5	Idempotent Process Call	Key `shopSystem + ":" + incrementId` , scope *global*, expiry 30 days; skips step 6–7 if this message was already processed to completion
6	Request-Reply OData `Lookup existing order`	`A_SalesOrder?$filter=<agreed reference field> eq '<shopSystem:incrementId>'&$select=SalesOrder` (field to be agreed with DyFlex, concept §8.4). Hit → take the number, skip creation. This covers the case "created but response lost" that the idempotent repository cannot
7	Mapping + Request-Reply OData `Create sales order`	OData V2 `API_SALES_ORDER_SRV` deep insert (D-20): header + items with shipping point per item (D-31), carrier at header (D-13), payment reference, requested delivery date, texts. Timeout 30 s
8	Groovy `Evaluate S/4 result`	HTTP 201 → sales order number; S/4 `E` message → PERMANENT_BUSINESS `S4_BUSINESS_ERROR` with message text; `A` / `W` handling per agreement (concept §8.4)

#	Step	Detail
9	Request-Reply HTTP `POST order comment`	`POST /V1/orders/<entityId>/comments` with `SAP order <no>` or `Not exported: <code> <text>` (A4-D14). Failure here is TRANSIENT but must **not** recreate the order – steps 5–6 guarantee that on retry
10	End	JMS message acknowledged, MPL custom status `AU_A4_CREATED`
E	Exception subprocess	`classifyError` → router: **TRANSIENT and retryCount < maxRetries** → Error End (the JMS adapter rolls back and redelivers with backoff; nothing else to do) · **PERMANENT or retries exhausted** → JMS receiver `AU_<STG>_DLQ` (properties `interfacename`, `errorClass`, `errorCode`, `errorText`, `retryCount`, original body) → HTTP `POST order comment` "Not exported: …" (best effort, own try/catch) → ANS alert → MPL custom status `AU_A4_DEAD_LETTERED` → End (normal, so the IN message is acknowledged in the same transaction as the DLQ write)

Transient errors retry with backoff; everything else is dead-lettered and alerted



Consumer error handling · retry loop, dead letter, alert, acknowledge

Only a transient error with retries left leaves the process through an Error End, which makes the JMS adapter roll back and redeliver after the backoff interval. Every other path ends normally, so the staging message is acknowledged in the same transaction that wrote the dead-letter entry.

Reprocessing. An operator with write access (purplecat) fixes the cause (e.g. missing SAP number maintained by the customer monitor), then deploys `GenericIS_ReprocessDLQ` for the stage. It reads the DLQ with one concurrent process and calls the owning consumer through ProcessDirect `/hydac/au/sls/<interfacename>/v1/reprocess`, which enters the same main process with `retryCount` reset. Messages that fail again go back to the DLQ; the operator undeploys the flow afterwards. Single entries can be deleted in *Manage Message Queues* when the business decides not to export an order.

What A1–A3 change: the consumer pulls company / customer / address instead of the order and writes to the customer monitor's CAP service (HTTP, same transient/permanent classification) instead of S/4; the approval in the monitor and the later `customer changed` event from S/4 are outside this chain (concept §6.3). Preconditions, idempotency key (`shopSystem:entityId`), dead letter and reprocessing are identical.

What pattern B reuses: `BusinessPartner_Notification2DataEvent` and `BusinessPartner2Webshop_<SHOP>` use the same consumer skeleton with the AMQP → JMS bridge per target shop system, the same error classes, and `AU_<STG>_DLQ` as the shared dead-letter queue; bulk writes additionally keep the changelog and poll `/V1/bulk/{uuid}/status` (concept §6.4).

Queue, package, naming and credential plan

AU needs 6 JMS queues in the non-production tenant (3 per stage) and 3 in production, which leaves 12 of the 18 free queues for US, DE and TPM. Event Mesh queues are not subject to this limit; they carry the per-shop-system multiplex.

JMS queues (Cloud Integration)

Queue	Stage	Written by	Read by	Purpose
`AU_DEV_WebshopToSAP_IN` / `AU_TST_WebshopToSAP_IN`	DEV / TST	`Webshop_Event_EM2JMS`	`Webshop_Event_Dispatch` → A1–A4	Staging with retry/backoff for shop → SAP

Queue	Stage	Written by	Read by	Purpose
`AU_DEV_SAPToWebshop_IN` / `AU_TST_SAPToWebshop_IN`	DEV / TST	B bridges (one per target shop)	B dispatcher → B1–B6	Staging for SAP → shop; `targetShop` property selects endpoint and credentials
`AU_DEV_DLQ` / `AU_TST_DLQ`	DEV / TST	every AU consumer's exception subprocess	`GenericIS_ReprocessDLQ` (on demand)	Dead letter for both directions; `interfacename` names the owner
`AU_PRD_WebshopToSAP_IN`, `AU_PRD_SAPToWebshop_IN`, `AU_PRD_DLQ`	PRD (`hydac-is-prod`)	as above	as above	Same layout, one stage

JMS settings for all `_IN` queues: expiration 30 days, retry 5 min with exponential backoff to 240 min, DLQ flag on, non-exclusive, 1 concurrent process per consumer (raise per flow after the load test). `_DLQ`: expiration 90 days, no consumer deployed by default.

Event Mesh (message client `AU_<STG>_CI` for Cloud Integration, one client per connected shop system is not needed because the shop never publishes, D-38)

Object	Name	Note
Topic shop → BTP	`ci/au/shop/<entity>/v1/<event>` e.g. `ci/au/shop/order/v1/export-requested`	Published by the generic inbound
Queue shop → BTP	`AU_<STG>_WebshopToSAP`	Subscribes `ci/au/shop/*/v1/*`; DMQ on, max redelivery 3 (the JMS layer does the real retrying)
Topic S/4 → BTP	`ce/sap/s4/beh/businesspartner/v1/BusinessPartner/*` etc.	Published by S/4 (concept §7.4)
Queues S/4 → BTP	`AU_<STG>_BusinessPartnerNotification`	One per stage
Topic BTP → shops	`ci/au/s4/<entity>/v1/<event>`	Published once by `*_Notification2DataEvent`
Queues BTP → shops	`AU_SHL_BusinessPartnerData`, `AU_STG_…`, `AU_TST_…`, `AU_DEV_…`	One per connected shop system (D-37, Q-20 answered by name)

Packages and iFlow IDs

Package	iFlow (name · ID)
`AU-SLS-GenericIS_Artifacts`	`AU_SLS_Common` script collection · `AU_ConditionTypes` value mapping · `GenericIS_ReprocessDLQ` · `com.hydac.AU.SLS.GenericIS_ReprocessDLQ` · `Webshop_Event_Dispatch` · `com.hydac.AU.SLS.Webshop_Event_Dispatch`
`AU-SLS-WebshopSyncAPIs`	`S1_CustomerPrices_API` + `S1_CustomerPrices_S4PC` … `S5_Freight_*` · `com.hydac.AU.SLS.S<n>_<Name>_<Layer>`
`AU-SLS-WebshopToSAP`	`Webshop_Event_Inbound`, `Webshop_Event_EM2JMS`, `A1_Company_Event2Monitor`, `A2_Contact_Event2Monitor`, `A3_Address_Event2Monitor`, `A4_Order_Event2SalesOrder`
`AU-SLS-SAPToWebshop`	`B1_BusinessPartner_Notification2DataEvent`, `B1_BusinessPartner2Webshop`, `B2_Stock2Webshop`, …

Stage copies carry the suffix `_DEV` / `_TST` in name and ID (concept §8.9, naming convention open point 5) and differ only in externalised parameters. The contract version `v1` is part of the endpoint path, not of the iFlow name, so a `v2` contract is a new contract iFlow next to the old one.

Security material

Alias	Type	Used by
`AU_S4_TST`, `AU_S4_PRD`	User credentials or client certificate (DyFlex communication user)	Backend iFlows, A4, B flows
`AU_Magento_DEV`, `AU_Magento_TST`, `AU_Magento_STG`, `AU_Magento_SHL`, `AU_Magento_PRD`	OAuth2 client credentials or token (TechDivision integration user per shop system, Q-04)	A pulls, B writes, order comments

Alias	Type	Used by
`AU_EM_<STG>`	OAuth2 client credentials of the Event Mesh message client	AMQP sender/receiver
`AU_ANS_<STG>`	OAuth2 client credentials of the Alert Notification Service instance	Dead-letter alerts
`AU_Webshop_Send`	User role on the process integration runtime instance for the AU shop client	HTTPS senders of all AU endpoints

Monitoring, alerting and security

Every failure is visible within one minute to the party that can act on it: business failures to HYDAC AU via Alert Notification Service e-mail, technical ones to purplecat, and all of them searchable in *Monitor → Integrations and APIs* by order number without BTP write access.

Concern	Mechanism
Find a message by business key	Custom header properties on every run: `shopSystem`, `websiteCode`, `entity`, `entityId`, `incrementId`, `correlationId`, `sapDocument`, `errorClass`, `errorCode`; `SAP_ApplicationID` = correlation id (searchable as *Application Message ID*)
Status at a glance	Custom status per outcome: `AU_SYNC_OK`, `AU_SYNC_FAILED_TRANSIENT`, `AU_SYNC_FAILED_PERMANENT`, `AU_A4_CREATED`, `AU_A4_RETRY`, `AU_A4_DEAD_LETTERED`, `AU_A4_SKIPPED_DUPLICATE`
Payload for analysis	Log attachments only in the exception subprocess and only on `PERMANENT` (no customer data in logs on the happy path; GDPR / Australian Privacy Act, Q-14); log level *Info* in TST, *None* for the sync flows in PRD

Concern	Mechanism
Alerting	`AU_SLS_Common.notify` posts an ANS event (`AU.Webshop.<interface>.<errorClass>` , severity by class) from the exception subprocess; ANS routes by class: business → HYDAC AU + HYDAC Digital mailbox, technical → purplecat; a second ANS rule alerts when `AU_<STG>_DLQ` entries > 0 for 15 min (via the JMS queue monitoring or a scheduled `GenericIS_QueueWatch` flow using the OData monitoring API)
Sync SLA	ANS/Cloud ALM threshold on error rate and P95 of the S-flows; input to the hosting decision D-25
Read access for HYDAC Digital / TechDivision	Access policy `AP_AU_Developer` covers all `AU-` artefacts (Q-12); request a separate read-only policy `AP_AU_Shop_Reader` on the three shop packages if HYDAC IT wants to keep `AU-EDI-*` separate
Shop → BTP auth	OAuth2 client credentials; a dedicated service key per shop system with role `AU_Webshop_Send` ; the US endpoints stay on their own role
BTP → S/4 auth	Communication arrangement per scenario with technical user `AU_S4_<STG>` (basic auth as DyFlex sets it up today; move to X.509 client certificate when DyFlex supports it)
BTP → Magento auth	Integration user per connected shop system with ACL limited to orders, companies, customers, inventory, prices and the HYDAC endpoints (concept §7.2)
Data at rest	JMS and DLQ payloads live encrypted in the tenant; DLQ expiry 90 days; no Data Store copies of orders

Decisions needed and next steps

Five decisions block the build; everything else can start on stage DEV against the Dev shop system (D-39) with the mock service for S/4.

#	Decision	Owner	Proposal
1	JMS queue budget for AU (6 non-prod + 3 prod) and queue naming under governance rule 2	HYDAC IT	Approve the plan above; one IN queue per direction and stage, one DLQ per stage
2	Shop → BTP authentication (Q-04)	purplecat, TechDivision	OAuth2 client credentials, one service key per shop system, role `AU_Webshop_Send`
3	S/4 reference field for the shop order number (idempotency, A4) and handling of `A` / `W` messages	DyFlex, purplecat	Custom field `YY1_ShopOrderRef` on the sales order header, value `<shopSystem>:<incrementId>`; `W` logged, `A` treated as error
4	Released APIs per interface (Q-01, IDD) – S1 price vs. simulation (open question S1-T1)	DyFlex	Backend iFlow is API-agnostic; mapping starts once the IDD names the APIs
5	Alert routing (who receives business vs. technical alerts) and ANS instance	HYDAC Digital, HYDAC AU, purplecat	One ANS instance per tenant, two subscriptions by severity

Next steps

- [ ] Review this design with TechDivision and DyFlex (naming, contract, error codes) – purplecat
- [ ] Create `AU-SLS-GenericIS_Artifacts` with script collection, value mapping, dispatcher and reprocessing flow on stage DEV – purplecat
- [ ] Build `S1_CustomerPrices_API` + `_S4PC` against the mock, then against S/4 Test; feed the latency test (concept §10.2) – purplecat

- [ ] Build `Webshop_Event_Inbound`, `Webshop_Event_EM2JMS`, `A4_Order_Event2SalesOrder`; test retry, dead letter and reprocessing with S/4 Test switched off – purplecat
- [ ] Request the JMS queues, Event Mesh queues and access policy from HYDAC IT – purplecat / HYDAC IT
- [ ] Copy the templates to S2–S5 and A1–A3 once the interface documents are frozen